

Лабораторная работа № 7

Тема: Разработка REST API и CRUD-операций

Цель работы: Разработать полноценный REST API для ресурса products, реализовать операции Create, Read, Update и Delete с использованием архитектуры routes → controllers → services и проверить полный CRUD-цикл через HTTP-клиент.

Практическое задание

Продолжите проект **Internet Shop API**.

Необходимо реализовать REST API:

GET /api/products

GET /api/products/:id

POST /api/products

PATCH /api/products/:id

DELETE /api/products/:id

После выполнения лабораторной работы один ресурс products должен поддерживать полный набор CRUD-операций.

Последовательность выполнения

1. Проверьте структуру проекта

express-shop/

```
|— controllers/
|   |— productController.js
|— middleware/
|— routes/
|   |— productRoutes.js
|— services/
|   |— productService.js
|— app.js
|— package.json
```

2. Создайте исходные данные

В services/productService.js:

```
let products = [
  {
    id: 1,
    name: "Keyboard",
    price: 15000,
    category: "Accessories",
    inStock: true
  },
  {
    id: 2,
    name: "Mouse",
```

```

    price: 8000,
    category: "Accessories",
    inStock: true
  },
  {
    id: 3,
    name: "Monitor",
    price: 85000,
    category: "Monitors",
    inStock: false
  }
];

```

3. Реализуйте получение всех товаров

```

export function getAllProducts() {
  return products;
}

```

4. Реализуйте получение товара по id

```

export function findProductById(id) {
  return products.find(
    product => product.id === id
  );
}

```

5. Реализуйте создание товара

```

export function addProduct(data) {
  const product = {
    id: products.length + 1,
    ...data
  };

  products.push(product);

  return product;
}

```

6. Реализуйте частичное изменение товара

```

export function updateProductById(id, changes) {
  const product = products.find(
    product => product.id === id
  );

  if (!product) {

```

```
    return null;
  }

  Object.assign(product, changes);

  return product;
}
```

7. Реализуйте удаление товара

```
export function deleteProductById(id) {
  const index = products.findIndex(
    product => product.id === id
  );

  if (index === -1) {
    return null;
  }

  const [deletedProduct] = products.splice(index, 1);

  return deletedProduct;
}
```

8. Импортируйте Service в Controller

B controllers/productController.js:

```
import {
  getAllProducts,
  findProductById,
  addProduct,
  updateProductById,
  deleteProductById
} from "../services/productService.js";
```

9. Реализуйте GET всех товаров

```
export function getProducts(req, res) {
  const products = getAllProducts();

  res.json(products);
}
```

Проверьте:

GET /api/products

Ожидаемый статус:

200 OK

10. Реализуйте GET одного товара

```
export function getProductById(req, res) {  
  const id = Number(req.params.id);  
  
  const product = findProductById(id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: "Product not found"  
    });  
  }  
  
  res.json(product);  
}
```

Проверьте:

GET /api/products/1

И:

GET /api/products/999

Второй запрос должен вернуть:

404 Not Found

11. Реализуйте POST

```
export function createProduct(req, res) {  
  const product = addProduct(req.body);  
  
  res.status(201).json(product);  
}
```

Отправьте:

POST /api/products

Тело:

```
{  
  "name": "SSD 1TB",  
  "price": 45000,  
  "category": "Components",  
  "inStock": true  
}
```

Ожидаемый статус:

201 Created

Запомните id созданного товара.

12. Реализуйте PATCH

```
export function updateProduct(req, res) {  
  const id = Number(req.params.id);
```

```
const product = updateProductById(  
  id,  
  req.body  
);
```

```
if (!product) {  
  return res.status(404).json({  
    message: "Product not found"  
  });  
}
```

```
res.json(product);  
}
```

Отправьте:

PATCH /api/products/4

Тело:

```
{  
  "price": 42000,  
  "inStock": false  
}
```

Убедитесь, что название и категория товара не изменились.

13. Реализуйте DELETE

```
export function deleteProduct(req, res) {  
  const id = Number(req.params.id);
```

```
  const product = deleteProductById(id);
```

```
  if (!product) {  
    return res.status(404).json({  
      message: "Product not found"  
    });  
  }
```

```
  res.json(product);  
}
```

Проверьте:

DELETE /api/products/4

14. Настройте Product Router

В routes/productRoutes.js:

```
import express from "express";
```

```
import {
```

```
getProducts,  
getProductById,  
createProduct,  
updateProduct,  
deleteProduct  
} from "../controllers/productController.js";
```

```
const router = express.Router();
```

```
router.get("/", getProducts);  
router.get("/:id", getProductById);  
router.post("/", createProduct);  
router.patch("/:id", updateProduct);  
router.delete("/:id", deleteProduct);
```

```
export default router;
```

15. Проверьте подключение Router

В app.js:

```
import express from "express";  
import productRoutes from "../routes/productRoutes.js";
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.use("/api/products", productRoutes);
```

```
app.listen(3000, () => {  
  console.log("Server started on port 3000");  
});
```

16. Выполните полный CRUD-цикл

Выполняйте запросы строго последовательно.

Create:

POST /api/products

Создайте новый товар и запишите его id.

Read:

GET /api/products/:id

Убедитесь, что созданный товар существует.

Update:

PATCH /api/products/:id

Измените цену и наличие.

Read:

GET /api/products/:id

Убедитесь, что изменения применились.

Delete:

DELETE /api/products/:id

Удалите товар.

Read после удаления:

GET /api/products/:id

Ожидаемый результат:

404 Not Found

Дополнительное задание

Реализуйте аналогичный REST API для ресурса categories:

GET /api/categories

GET /api/categories/:id

POST /api/categories

PATCH /api/categories/:id

DELETE /api/categories/:id

Используйте отдельные файлы:

routes/categoryRoutes.js

controllers/categoryController.js

services/categoryService.js

Требования к результату

Приложение должно:

- использовать ресурсные URL;
- поддерживать GET списка товаров;
- поддерживать GET одного товара;
- поддерживать POST;
- поддерживать PATCH;
- поддерживать DELETE;
- возвращать 201 после создания товара;
- возвращать 404 для отсутствующего товара;
- использовать разделение Route → Controller → Service;
- не передавать req и res в Service;
- возвращать данные в JSON;
- успешно проходить полный CRUD-цикл.